File Descriptor (M. Harshika - 2520090047)

- A **file descriptor (FD)** is simply an integer number that the operating system gives to a process when it opens something.
- The operating system assigns the lowest available file descriptor number when you open a resource.

Think of it as a ticket number.

Suppose you tell Linux:

> Open **student.txt**

Linux may say:

> Okay, I will give you ticket number 3.

So:

Student.txt —-> FD3

Your program doesn't have to keep saying "**student.txt**" every time.

It can simply say FD 3 to refer to that open file.

Similarly, Linux gives your program a number:

File → File Descriptor

For example: *student.txt → 3*

That number 3 is the file descriptor.

Every normal process starts with three standard file descriptors:

0 → Standard Input

1 → Standard Output

2 → Standard Error

0 → stdin → Where do I GET input?

1 → stdout → Where do I SEND normal output?

2 → stderr → Where do I SEND error output?

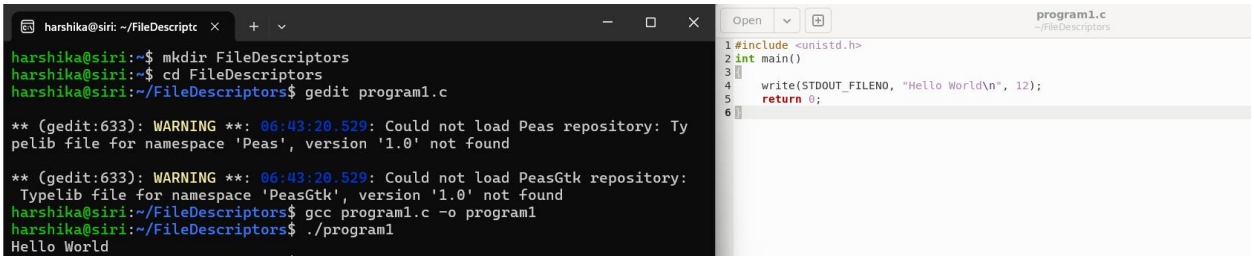
We normally write them as:

STDIN_FILENO
STDOUT_FILENO
STDERR_FILENO

Number	Name	Meaning
0	STDIN_FILENO	Input
1	STDOUT_FILENO	Output
2	STDERR_FILENO	Error output

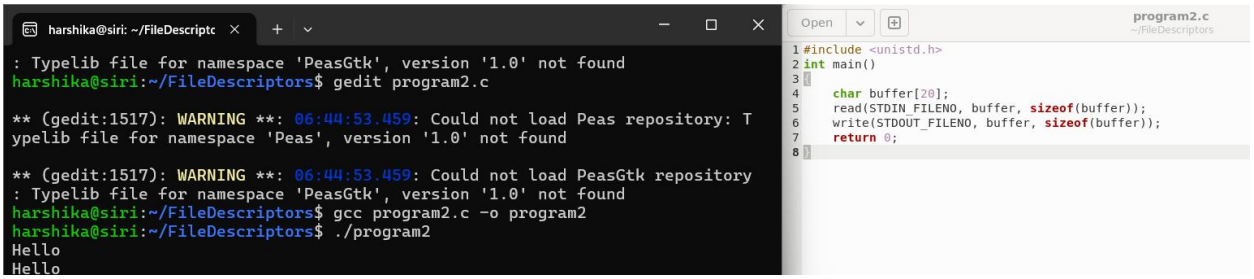
Program 1: stdout — Standard Output

```
#include <unistd.h>
int main()
{
   write(STDOUT_FILENO, "Hello World\n", 12);
   return 0;
}
```



Program 2: stdin — Standard Input

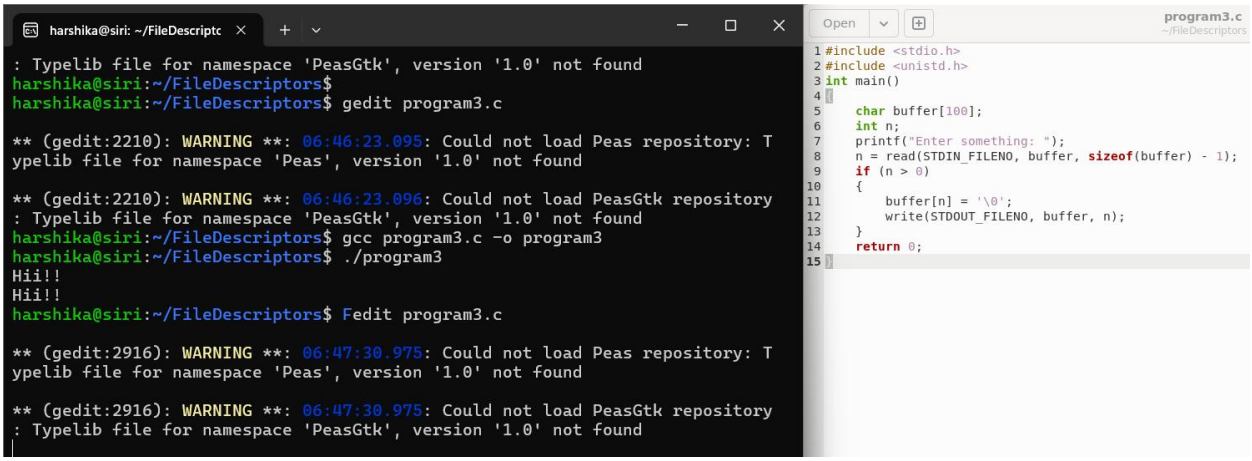
```
#include <unistd.h>
int main()
{
   char buffer[20];
   read(STDIN_FILENO, buffer, sizeof(buffer));
   write(STDOUT_FILENO, buffer, sizeof(buffer));
   return 0;
}
```



Program 3: Proper stdin + stdout

```
#include <stdio.h>
#include <unistd.h>
int main()
{
   char buffer[100];
   int n;
   printf("Enter something: ");
   n = read(STDIN_FILENO, buffer, sizeof(buffer) - 1);
   if (n > 0)
```

```
    {
        buffer[n] = '\0';
        write(STDOUT_FILENO, buffer, n);
    }
    return 0;
}
```



Program 4: stderr — Standard Error

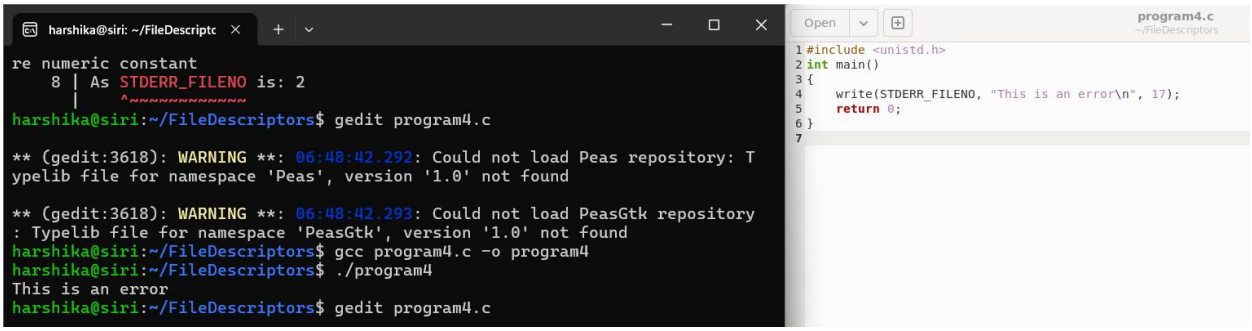
```
#include <unistd.h>
int main()
{
    write(STDERR_FILENO, "This is an error\n", 17);
    return 0;
}
```

As STDERR_FILENO is: 2

write(STDERR_FILENO, "This is an error\n", 17) is equivalent to:

write(2, "This is an error\n", 17);

You will see output as: This is an error.



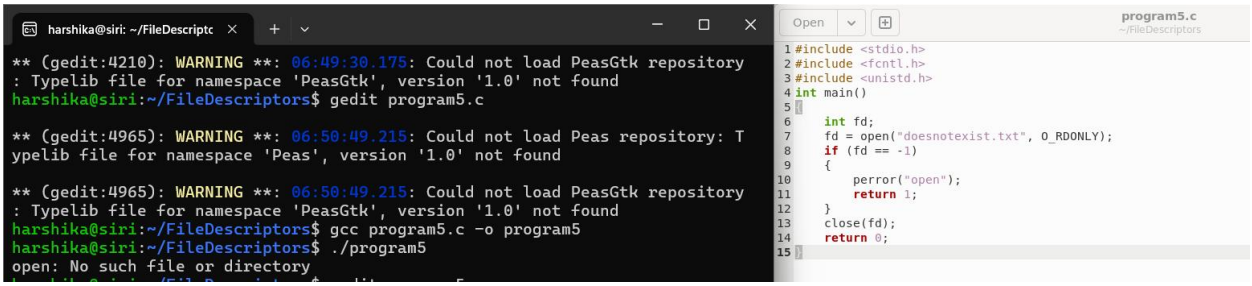
Program 5: stderr for an Actual Error

```
#include <stdio.h>

#include <fcntl.h>

#include <unistd.h>

int main()
{
   int fd;
   fd = open("doesnotexist.txt", O_RDONLY);
   if (fd == -1)
   {
      perror("open");
      return 1;
   }
   close(fd);
   return 0;
}
```



Program 6 Creating file using file descriptor

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
   int fd;
   fd = open("student.txt", O_CREAT | O_WRONLY, 0644);
   if (fd == -1)
   {
      perror("open");
      return 1;
   }
   printf("File created successfully!\n");
   printf("File descriptor = %d\n", fd);
   close(fd);
   return 0;
}
```

gcc create.c -o create
./create

Output:
File created successfully! File descriptor = 3

fd = open("student.txt", O_CREAT | O_WRONLY, 0644); describes the information below:

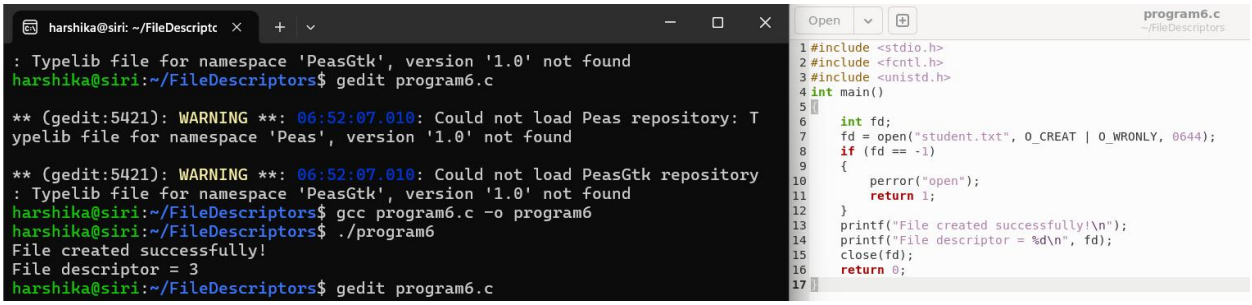
"student.txt" → file name

O_CREAT → create if it doesn't exist

O_WRONLY → open for writing

0644 → file permissions

fd → stores the file descriptor



Program 7

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
   int fd;
   fd = open("student.txt", O_WRONLY | O_TRUNC);
   if (fd == -1)
   {
      perror("open");
      return 1;
   }
   write(fd, "New Data\n", 9);
   close(fd);
   return 0;
}
```

If a file already has data, O_TRUNC removes all its existing contents when you open it for writing.

First, suppose student.txt contains:

Hello
Welcome

After open() with O_TRUNC:
Student.txt contains (empty)

DUP ( ) : duplicate a file descriptor.

dup() creates a new file descriptor that refers to the same underlying open file or I/O resource as the original descriptor.

Suppose:

int fd1 = open("student.txt", O_WRONLY);

If the OS gives:

fd1 = 3

then:

FD 3 → student.txt

Now we do:

int fd2 = dup(fd1);

The OS creates another file descriptor, usually:

fd2 = 4

Now both FD 3 and FD 4 refer to the same open file.

Program 7 - Understand dup()

```
#include <stdio.h>
#include <fcntl.h>
```

```c
#include <unistd.h>
int main()
{
   int fd1, fd2;
   fd1 = open("student.txt", O_CREAT | O_WRONLY, 0644);
   if (fd1 == -1)
   {
      perror("open");
      return 1;
   }
   printf("fd1 = %d\n", fd1);
   fd2 = dup(fd1);
   printf("fd2 = %d\n", fd2);
   close(fd1);
   close(fd2);
   return 0;
}
```

Output: fd1 = 3 fd2 = 4